

=== PROJET: NutriTrack - Application Suivi Nutritionnel Intelligent ===

CONTEXTE:

Tu dois développer une application mobile (React Native) complète de suivi alimentaire et nutritionnel. L'app doit aider les utilisateurs à prendre conscience de leur alimentation, atteindre leurs objectifs santé, et rester motivés via un système gamifié avec personnage animé.

SECTION 1: DONNÉES & UTILISATEUR

L'application collecte LORS DE L'ONBOARDING:

A. Infos de Base:

- Sexe (M/F/Autre)
- Âge (entier)
- Poids actuel en kg (décimal)
- Taille en cm (entier)
- Poids objectif en kg (décimal)
- Délai pour atteindre objectif (semaines) - affecte déficit/surplus recommandé
- Niveau d'activité: Sédentaire / Léger / Modéré / Actif / Très Actif

B. Santé & Restrictions:

- Liste conditions existantes (diabète, hypertension, etc.) - string libre ou checkboxes
- Allergies alimentaires (liste)
- Intolérances (gluten, lactose, etc.)
- Régime: Omnivore / Végétarien / Végan / Pescatarian / Autre
- Cuisines préférées (pour recommandations)
- Aliments à éviter (texte libre)

Cette data est stockée et utilisée pour:

- Calcul BMR (Basal Metabolic Rate) via Mifflin-St Jeor
- Calcul TDEE (Total Daily Energy Expenditure) = BMR * activité
- Calcul déficit/surplus calorique idéal
- Filtrage recommandations alimentaires
- Personnalisation seuils nutritionnels

SECTION 2: FONCTIONNALITÉS PRINCIPALES

FONCTIONNALITÉ 1: DASHBOARD QUOTIDIEN (écran principal)

Affichage:

- En haut: Date du jour + bouton jour précédent/suivant
- Section PERSONNAGE ANIMÉ:
 - * SVG/Lottie animé représentant l'utilisateur

- * Remplissage du personnage = % calories consommées/objectif
- * Couleurs: vert (bon) → jaune (warning) → rouge (surplus)
- * Animations:
 - Heureux si équilibre nutritionnel bon
 - Triste si trop de sucre/sodium
 - Malade si dépassement calorique important
- * Texte motivant dynamique selon situation

- Section CALORIES:
 - * Consommé: XXX / Objectif: XXXX kcal
 - * Reste: XXX kcal
 - * Progress bar visuelle

- Section MACRONUTRIMENTS (3 barres):
 - * Protéines: XXg / XXg recommandé (visuel couleur)
 - * Glucides: XXg / XXg recommandé
 - * Lipides: XXg / XXg recommandé
 - * Code couleur: vert si dans zone, orange si légèrement haut, rouge si critique

- Section SANTÉ (mini-indicateurs):
 - * Fibres: XXg / 25g idéal
 - * Sucres: XXg / 50g seuil (configurable par condition)
 - * Sodium: XXmg / 2300mg seuil
 - * Cholestérol: si pertinent
 - * Score couleur pour chaque (A-F notation)

- Section ALIMENTS JOUR:
 - * Liste scrollable repas ajoutés
 - * Par repas: nom + calories + macro rapide
 - * Icône éditer / supprimer par aliment
 - * Bouton "+" pour ajouter nouveau (modal choix saisie)

- Boutons d'action (bas écran):
 - *  Photo
 - *  Voix
 - *  Manuel

INTERACTIONS:

- Tap personnage = voir conseils du jour
- Tap jour précédent/suivant = consulter historique
- Swipe haut = voir analytics complètes
- Long-press aliment = options édition

FONCTIONNALITÉ 2: SAISIE PAR PHOTO

Flow:

1. Écran caméra (preview temps réel)
2. Photo capturée → envoyée API vision (TensorFlow.js côté client OU serveur)
3. IA détecte:
 - Objets alimentaires identifiés (ex: "poulet rôti", "riz", "salade")
 - Estimation portions (volume/poids approximatif)
 - Confiance détection (%)
4. Écran confirmation/édition:
 - Affiche détections trouvées
 - Chaque aliment: nom + portion détectée
 - User peut: corriger, ajouter, supprimer items
 - Affiche calories/macros temps réel
 - Validation = ajout au jour

DÉTAILS TECHNIQUES:

- Modèle IA: TensorFlow Lite (détection objets) + base aliments
- Estimation portion: ratio pixel → dimensions réelles (optionnel: demander main pour échelle)
- Si confiance < 60%: demander confirmation manuelle
- Sauvegarde image de référence (optionnel, avec consentement)

FONCTIONNALITÉ 3: SAISIE PAR VOIX

Flow:

1. Tap microphone → enregistrement audio
2. Transcription temps réel ou après stop
3. Parsing intelligent du texte:
 - Extraction aliments (NLP)
 - Extraction portions ("une poignée", "1 bol", "200g")
 - Extraction répétitions ("2 oeufs", "3 verres")
4. Suggère aliments détectés
5. User valide/corrige
6. Sauvegarde

EXEMPLES INPUTS:

- "J'ai mangé une tranche de pain complet avec du beurre et une tomate"
→ pain complet (30g) + beurre (10g) + tomate (100g)
- "Deux oeufs brouillés avec du fromage râpé"
→ œufs (2x50g) + fromage (20g)
- "Un verre de jus d'orange"
→ jus orange (250ml)

DÉTAILS TECHNIQUES:

- Service: Web Speech API (gratuit) OU Deepgram/Google Cloud Speech
- Parser: Regex + NLP léger OU LLM (Gemini mini) pour interprétation
- Fallback: si parsing échoue, liste aliments pour sélection manuelle

FONCTIONNALITÉ 4: SAISIE MANUELLE

Flow:

1. Écran saisie simple:

- Champ recherche aliment (autocomplétion)
- Dropdown catégories (petit-déjeuner, déjeuner, etc.)
- Champs: quantité + unité (g, ml, portion, bol, verre, etc.)
- Raccourcis favoris (clic 1 = reprise dernier repas)

2. Suggestion aliments récents/favoris

3. Validation = ajout au jour

BASE ALIMENTS:

- Intégration OpenFoodFacts API (gratuit, 400k+ aliments)
- + USDA FoodData Central (USA)
- Cache local (top 500 aliments)
- Fallback: import nutrition manuel si absent

FONCTIONNALITÉ 5: ÉDITION/SUPPRESSION

- User peut éditer n'importe quel aliment ajouté:

- * Tap → modal édition avec champs modifiables
- * Recalcul auto des nutritionnels
- * Sauvegarde local puis sync serveur

- Suppression: swipe ou bouton delete

- * Confirmation avant suppression
- * Historique sauvegardé (audit trail)

FONCTIONNALITÉ 6: ANALYTICS & TENDANCES

Écran dédié (swipe ou menu):

A. Graphiques Période:

- Toggle 7j / 30j / 90j
- Graphique linéaire calories consommées vs objectif (zone de confiance)
- Graphique barres macros moyennes
- Tendance (pente pour estimer progression)

B. Radar Nutritionnel:

- 6 axes: calories, protéines, glucides, lipides, fibres, micronutriments
- Comparer semaine actuelle vs semaine précédente
- Zone idéale en fond (contexte)

C. Statistiques:

- Jours/semaine objectif atteint (%)
- Aliments favoris mangés
- Macros moyennes
- Score santé global (A-F)

D. Prédictions:

- Si maintien trend: estimation poids dans 4 semaines
- Atteinte objectif: oui/non, date estimée

E. Anomalies & Alertes:

- "Dépassement sodium 3x cette semaine"
- "Sucres 2x plus que recommandé lundi/mardi"
- "Protéines insuffisantes mardi-jeudi"

SECTION 3: SYSTÈME RECOMMANDATIONS PERSONNALISÉES

L'app génère recommandations basées sur:

1. Déficits détectés:

- Si protéines < seuil: "Ajoute du poulet, oeufs, ou lentilles pour protéines"
- Si fibres < 25g: "Manque de fibres! Essaie riz complet, brocoli, pommes"

2. Excès détectés:

- Si sucre élevé: "Sucre élevé cette semaine, réduis boissons sucrées"
- Si sodium > 2300mg: "Attention sodium! Limite sauces et produits transformés"

3. Suggestions repas équilibrés:

- Basées sur préférences + restrictions
- Exemple: "Pour dîner équilibré 600 kcal: poulet 150g + riz 150g + brocoli 200g"

4. Pattern recognition:

- Si user déjasse souvent le WE: "Attention au WE, trend sucre+calories ↑"
- Si déficit trop important: "Déficit calorique trop agressif, mange 200 kcal plus"

SECTION 4: ONBOARDING UTILISATEUR

Écrans séquentiels:

1. Welcome slide (app vision)
2. Profil de base (nom, âge, sexe, photo optionnelle)
3. Objectifs (poids, délai, activité)

4. Conditions santé (checkboxes conditions, allergies text)
 5. Préférences alimentaires (régime, cuisines, restrictions)
 6. Résumé + édition rapide
- Calcul automatique besoins nutritionnels
 - Intro personnage animé
 - Accès dashboard

SECTION 5: ARCHITECTURE DONNÉES

MODÈLES:

User:

```
{
  id: UUID,
  email: string,
  nom: string,
  sexe: "M" | "F" | "Autre",
  age: int,
  poids_actuel_kg: float,
  taille_cm: int,
  poids_objectif_kg: float,
  delai_semaines: int,
  niveau_activite: "sedentaire" | "leger" | "modere" | "actif" | "tres_actif",
  conditions_sante: string[],
  allergies: string[],
  intolerances: string[],
  regime: string,
  cuisines_preferrees: string[],
  aliments_a_eviter: string[],
  created_at: date,
  updated_at: date,

  // Computed fields:
  bmr: float (Mifflin-St Jeor),
  tdee: float (BMR * activité),
  besoin_calories_jour: float,
  seuils_nutritionnels: {
    proteines_min: float,
    glucides_min: float,
    lipides_min: float,
    sucre_max: float,
    sodium_max: float,
    fibres_min: float
  }
}
```

MealEntry:

```
{
  id: UUID,
  user_id: UUID,
  date: date,
  time: time,
  aliments: Aliment[] (array),
  repas_type: "petit-dej" | "dejeuner" | "gouter" | "diner" | "snack",
  source: "photo" | "voix" | "manuel",
  photo_url: string (optionnel),
  created_at: date,
  updated_at: date
}
```

Aliment:

```
{
  id: UUID,
  meal_entry_id: UUID,
  nom: string,
  quantite: float,
  unite: "g" | "ml" | "portion" | "bol" | "verre" | etc,
  calories: float,
  proteines_g: float,
  glucides_g: float,
  lipides_g: float,
  fibres_g: float,
  sucres_g: float,
  sodium_mg: float,
  source: "openfoods" | "usda" | "manuel",
  food_id: string (reference externe)
}
```

DailyNutrition (computed view):

```
{
  user_id: UUID,
  date: date,
  calories_total: float,
  proteines_total: float,
  glucides_total: float,
  lipides_total: float,
  fibres_total: float,
  sucres_total: float,
  sodium_total: float,
  % vs objectif: float,
  score_sante: "A" | "B" | "C" | "D" | "F"
}
```

SECTION 6: INTERFACES À DÉVELOPPER

Wireframes avec interactions:

1. DASHBOARD (écran principal)
 - Header: date + nav prev/next
 - Personnage animé centered (SVG Lottie)
 - Infos calories/macros
 - Indicateurs santé mini
 - Liste aliments scroll
 - Boutons saisie fixed bottom
2. CAMERA SCREEN
 - Caméra fullscreen preview
 - Overlay guide cadrage
 - Bouton photo centered bottom
3. CONFIRMATION PHOTO
 - Affiche aliments détectés en cartes
 - Chaque carte: edit/delete
 - Total calories preview
 - Bouton valider
4. VOICE SCREEN
 - Animated mic visualizer
 - Transcript en temps réel
 - Boutons Start/Stop/Clear
5. MANUAL INPUT SCREEN
 - Recherche aliment (autocomplete)
 - Champs quantité + unité
 - Favoris quick access
 - Validation bouton
6. ANALYTICS SCREEN
 - Tabs: Graphiques / Stats / Prédictions
 - Graphique calories trend
 - Radar nutritionnel
 - Anomalies list
7. SETTINGS SCREEN
 - Édition profil
 - Besoins nutritionnels (affichage)
 - Export PDF rapport
 - À propos

PALETTE COULEUR SUGGÉRÉE:

- Vert: #2EC4B6 (équilibre, santé)
- Orange: #FF9F1C (warning, modération)
- Rouge: #E63946 (dépassement)
- Bleu: #1D3557 (données, froid)
- Gris: #F1FAEE (fond neutre)

SECTION 7: TECHNOLOGIES RECOMMANDÉES

Frontend:

- React Native + Expo (multiplateforme rapide)
- OU React Native CLI + TypeScript (plus complet)
- State: Redux/Zustand (gestion état complexe)
- UI: React Native Paper / Native Base (components)
- Graphiques: React Native Chart Kit (gauche) OU Victory Native
- Animations: Lottie (personnage)
- Vision: TensorFlow Lite + react-native-camera

Backend:

- Node.js Express OU Python FastAPI
- Base données: PostgreSQL (robustesse) + Redis (cache)
- Auth: Firebase Auth OU JWT + bcrypt
- APIs: OpenFoodFacts API + USDA FoodData Central
- Vision API: TensorFlow.js serveur OU TensorFlow Serving
- Speech: Deepgram OU Google Cloud Speech-to-Text

Déploiement:

- Frontend: Expo OU Vercel/Firebase Hosting
- Backend: Heroku / AWS / DigitalOcean / Railway
- DB: PostgreSQL cloud (Supabase / Railway / AWS RDS)
- Storage: S3 pour photos

SECTION 8: PHASES DÉVELOPPEMENT RECOMMANDÉES

PHASE 1 - MVP (4-5 semaines):

- ✓ Onboarding user profil
- ✓ Dashboard visualisation calories/macros simple
- ✓ Saisie manuelle aliments (OpenFoodFacts)
- ✓ Édition/suppression aliments
- ✓ Authentification basique
- ✓ Stockage local données

PHASE 2 - Photo & Voix (3-4 semaines):

- ✓ Intégration caméra + détection IA
- ✓ Voix-texte transcription
- ✓ Parsing intelligent inputs

PHASE 3 - Analytics (2-3 semaines):

- ✓ Dashboard analytics/tendances
- ✓ Graphiques + radar nutritionnel
- ✓ Recommandations intelligentes

PHASE 4 - Gamification & Polish (2-3 semaines):

- ✓ Animations personnage améliorées
- ✓ Badges/défis (optionnel)
- ✓ Notifications motivantes
- ✓ UI/UX refinement

SECTION 9: QUESTIONS CLÉS POUR CLARIFIER

1. Plateforme prioritaire: iOS, Android, ou les deux?
2. Besoin authentification cloud ou offline-first?
3. Intégration Apple Health / Google Fit recommandée?
4. Base données aliments locale ou cloud?
5. Budget pour APIs externes (Deepgram, TensorFlow serving)?
6. Partage médecin: besoin vrai ou juste idée?
7. Niveau gamification désiré (simple vs complexe)?

BON DE COMMANDE RÉSUMÉ:

Développer une app React Native/mobile complète NutriTrack avec:

- Dashboard nutritionnel + personnage animé
- 3 modes saisie (photo IA, voix transcription, manuel)
- Calculs nutritionnels personnalisés (BMR/TDEE/macros)
- Analytics tendances 7/30/90j
- Recommandations intelligentes
- Belles interfaces motivantes (design included)
- Backend NodeJS + PostgreSQL
- Authentification + cloud sync
- Déploiement complet instructions

Budget estimé: 80-120 heures dev (agile, itérations)

Timeline: 3-4 mois si full-time, parties par phases possibles

"IMPORTANT - Approche Minimaliste:

- Information hiérarchisée (primaire → secondaire)
- Chaque écran max 3-4 éléments principaux
- Détails: accordéons/collapse plutôt que scrolls infinis
- Espaces blancs généreuses (30% de l'écran vide = respiration)
- Une action principale par écran (CTA unique ou groupé)
- Textes courts: max 2 lignes par info
- Pas de 'et aussi', 'aussi voir', 'plus d'info' → page dédiée
- Couleurs: max 3 (primaire + accent + neutre)
- Une métrique vedette par écran"

// NOUVELLE RECOMMANDATION TECH (adapté à toi):

Frontend:

- Flutter + Dart  (tu maîtrises)
- Riverpod (état management clean)
- GetIt (injection dépendances)
- BLoC pattern (architecture clean)

Backend:

- Reste Node.js Express  (contraste tech ok)
- OU Dart Shelf (même langage partout)

Vision IA:

- TensorFlow Lite (models offline)
- Google ML Kit (Flutter plugin native)

Audio:

- speech_to_text (plugin Flutter top)
- Google Cloud Speech (fallback)

Graphics:

- FL Chart (MEILLEUR pour Flutter)
- Heatmap pour nutritional radar

Animations:

- Lottie Flutter (parfait)

Prompt sécurité à ajouter:

"=== SÉCURITÉ OBLIGATOIRE ==="

1. AUTHENTIFICATION:

- ✓ JWT tokens (access + refresh)
- ✓ Refresh token rotation toutes les 7 jours
- ✓ Déconnexion efface tokens locaux
- ✓ Biometric auth (FaceID/TouchID) optionnel

2. STOCKAGE:

- ✓ flutter_secure_storage pour tokens (encrypto native iOS/Android)
- ✓ Hive + chiffrement pour données locales
- ✓ Pas de plaintext, jamais
- ✓ Données sensibles = volatile en RAM (token)

3. RÉSEAU:

- ✓ HTTPS obligatoire
- ✓ Certificate pinning pour API backend
- ✓ Timeouts requis (30s max)
- ✓ Validation SSL custom

4. DONNÉES UTILISATEUR:

- ✓ Poids/âge/santé = 🔒 chiffré en transit
- ✓ Photos aliments = supprimées après traitement IA
- ✓ GDPR compliant (export, delete data)
- ✓ Audit logs serveur (accès données)

5. INPUT VALIDATION:

- ✓ Validation client (quantités, types)
- ✓ Validation serveur (stricte)
- ✓ SQL injection protection (ORM)
- ✓ Rate limiting API (max 100 req/5min)

6. CODE:

- ✓ Secrets = variables env (jamais en git)
- ✓ API keys serveur-side (pas frontend)
- ✓ Analyse statique: dart analyze
- ✓ Dependencies updates mensuelles

"

=== PALETTE NutriTrack ===

- **PRIMAIRE** (Confiance, Santé)
Couleur: #27AE60 (Vert naturel)
Variantes:
 - Dark: #1E8449 (boutons, headers)
 - Light: #A9DFBF (backgrounds)
 - Pale: #D5F4E6 (cards background)

- **ACCENT** (Attention, Équilibre)
Couleur: #F39C12 (Orange doré)
Variantes:

- Dark: #C17C1B (warning forts)
- Light: #FCE5CD (highlight light)

- ALERT (Excès, Dépassement)
Couleur: #E74C3C (Rouge soft)
Variantes:
 - Dark: #A93226 (critical)
 - Light: #FADBD8 (error backgrounds)

- NEUTRALS (Fond, Texte)
 - Blanc: #FFFFFF (backgrounds)
 - Gris clair: #F8F9FA (cards)
 - Gris moyen: #95A5A6 (labels)
 - Gris foncé: #2C3E50 (texte principal)
 - Noir: #1A1A1A (headings)

- DONNÉES (Charts)
 - Protéines: #3498DB (bleu)
 - Glucides: #F39C12 (orange)
 - Lipides: #E74C3C (rouge)
 - Fibres: #27AE60 (vert)

=== MODE SOMBRE (optionnel) ===

Inversé intelligent (non juste négatif):

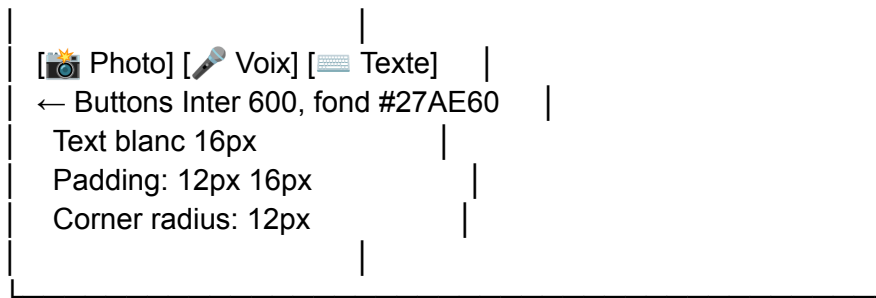
- Primaire: #2ECC71 (vert clair)
- Dark bg: #1A2D1A (vert très foncé)
- Cards: #234423
- Text: #ECF0F1

=== TYPOGRAPHIE ===

FONTS À UTILISER:

- Titre: Inter Bold (google_fonts)
 - * Weights: 600, 700
 - * Usage: Headers, titles
 - * Spacing: -0.5px (tight)
- Body: Poppins Regular (google_fonts)
 - * Weights: 400, 500
 - * Usage: Textes, labels
 - * Spacing: 0px
- Accent: Inter SemiBold (google_fonts)
 - * Weights: 600
 - * Usage: Buttons, emphasis

ÉCHELLE TYPOGRAPHIQUE:



ESPACES:

- Entre sections: 16px
- Padding écran: 16px côtés, 24px haut/bas
- Entre éléments: 8px

```
class AppColors {  
  // Primaires  
  static const primary = Color(0xFF27AE60);    // Vert  
  static const primaryDark = Color(0xFF1E8449);  
  static const primaryLight = Color(0xFFA9DFBF);  
  static const primaryPale = Color(0xFFD5F4E6);  
  
  // Accent  
  static const accent = Color(0xFFFF39C1);      // Orange  
  static const accentDark = Color(0xFFC17C1B);  
  static const accentLight = Color(0xFFFFCE5CD);  
  
  // Alerts  
  static const error = Color(0xFFE74C3C);       // Rouge  
  static const errorDark = Color(0xFFA93226);  
  static const errorLight = Color(0xFFFFADB8);  
  
  // Neutrals  
  static const textPrimary = Color(0xFF2C3E50);  
  static const textSecondary = Color(0xFF95A5A6);  
  static const bgLight = Color(0xFFFFF8F9FA);  
  static const bgDark = Color(0xFFFFFFFF);  
  
  // Charts  
  static const chartProtein = Color(0xFF3498DB);  
  static const chartCarbs = Color(0xFFFF39C1);  
  static const chartFats = Color(0xFFE74C3C);  
  static const chartFiber = Color(0xFF27AE60);  
}
```

=== DESIGN FINAL RECOMMANDÉ ===

Base: LifeFit (structure 50 screens)
Style: Coffee Minimalist (épuré, espacé)
Couleur: Vert #27AE60 + Orange accent (évite rose/bleu)
Typo: Inter + Poppins (minimaliste moderne)
Nom: VITAE (court, mémorable, élégant)

Palette: Vert (santé) + Orange (attention) + Blanc (respiration)
Theme: Naturel, apaisé, confiant (pas stressant)

=== SECTION DESIGN & STYLE ===

Inspiration visuelle:

- Base structure: LifeFit app (50 screens UI Kit)
- Style épuré: Coffee Minimalist app UI
- Palette: Vert naturel #27AE60 + Orange #F39C12

Nom App: VITAE

Tagline: "Vitae - Ton Journal Nutritionnel Intelligent"

Palette Couleur:

- Primaire: #27AE60 (vert naturel santé)
- Accent: #F39C12 (orange confiance)
- Erreur: #E74C3C (rouge soft)
- Neutres: #2C3E50 text, #F8F9FA bg, #FFFFFF cards

Typographie:

- Font: Inter Bold (headings), Poppins Regular (body)
- H1: 30px / H2: 24px / H3: 18px / Body: 16px / Label: 14px
- Minimaliste, très espacé, respiration

Principes Design:

- Épuré: max 3-4 éléments/écran
- Espaces blancs: 30% écran vide minimum
- Une action principale par page
- Accessibilité: WCAG AA minimum
- Animations: Lottie fluides, pas de loading spinners agressifs

Icones: Material Design 3 ou Feather Icons

Illustration: Personnage simple SVG animé (personnalisé par user profile)

// Firebase Firestore Rules (sécurisé)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/meals/{mealId} {
      allow read, write: if request.auth.uid == userId;
```



```

    // Seulement l'user peut accéder SES data
  }
  match /users/{userId} {
    allow read: if request.auth.uid == userId;
    allow write: if request.auth.uid == userId;
    // Surtout: JAMAIS les infos santé cross-user
  }
}
}
}

```

Backend Phase 1: Firebase (Firestore + Auth + Functions)
 Backend Phase 2+: Migration option Node.js + PostgreSQL

Raison: Maximum vitesse MVP, zéro ops, data sécurisée

Phase 1 (MVP):

- ✓ Google ML Kit (device) = détection aliments simple
- ✓ OpenFoodFacts API = nutrition basique
- ✓ User corrige portions manuellement

Phase 2+ (Nice to have):

- ✓ TensorFlow serveur = estimation portions auto
- ✓ ML Model fine-tuning = plus d'acuité
- ✓ Stockage photos = entraînement modèle custom

"Intégration Vision IA:

- Phase 1: Google ML Kit (object detection)
- Détecte: tomate, poulet, riz, etc.
- User estime portion, app cherche nutrition
- Fallback: OpenFoodFacts API

- Optionnel Phase 2: TensorFlow Lite
- serveur-side pour estimation portions

"

```

lib/
├── shared/
│   └── theme/
│       ├── app_colors.dart    ← TOUTES les couleurs
│       ├── app_typography.dart ← Toutes les typos
│       ├── app_theme.dart     ← ThemeData Flutter
│       └── theme_mode.dart    ← Light/Dark switch

```

```

abstract class AppColors {
  // PRIMAIRE - Vert
  static const Color primaryColor = Color(0xFF27AE60);
  static const Color primaryDark = Color(0xFF1E8449);
  static const Color primaryLight = Color(0xFFA9DFBF);

```

```

static const Color primaryPale = Color(0xFFD5F4E6);

// ACCENT - Orange
static const Color accentColor = Color(0xFFFF39C12);
static const Color accentDark = Color(0xFFC17C1B);
static const Color accentLight = Color(0xFFFFCE5CD);

// ALERTS
static const Color errorColor = Color(0xFFE74C3C);
static const Color errorDark = Color(0xFFA93226);
static const Color errorLight = Color(0xFFFFADB8);

static const Color successColor = Color(0xFF27AE60);
static const Color warningColor = Color(0xFFFF39C12);

// NEUTRALS
static const Color textPrimary = Color(0xFF2C3E50);
static const Color textSecondary = Color(0xFF95A5A6);
static const Color textTertiary = Color(0xFFBDC3C7);
static const Color bgLight = Color(0xFFFFF8F9FA);
static const Color bgDark = Color(0xFFFFFFFF);
static const Color borderColor = Color(0xFFECEFF1);

// CHARTS
static const Color chartProtein = Color(0xFF3498DB);
static const Color chartCarbs = Color(0xFFFF39C12);
static const Color chartFats = Color(0xFFE74C3C);
static const Color chartFiber = Color(0xFF27AE60);

// NUTRITION STATUS
static const Color nutritionGood = Color(0xFF27AE60); // Vert
static const Color nutritionWarning = Color(0xFFFF39C12); // Orange
static const Color nutritionBad = Color(0xFFE74C3C); // Rouge
}

// DARK MODE (optionnel future)
abstract class AppColorsDark {
    static const Color primaryColor = Color(0xFF2ECC71);
    static const Color bgDark = Color(0xFF1A2D1A);
    // ... etc
}

class AppTheme {
    static ThemeData lightTheme() {
        return ThemeData(
            useMaterial3: true,

```

```

// Couleurs
primaryColor: AppColors.primaryColor,
scaffoldBackgroundColor: AppColors.bgDark,

// Boutons
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: AppColors.primaryColor,
    foregroundColor: Colors.white,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    padding: EdgeInsets.symmetric(vertical: 16),
  ),
),

// Text theme (utilise app_typography.dart)
textTheme: TextTheme(
  displayLarge: AppTypography.h1,
  displayMedium: AppTypography.h2,
  titleLarge: AppTypography.h3,
  bodyLarge: AppTypography.body,
  labelSmall: AppTypography.label,
),

// Input
inputDecorationTheme: InputDecorationTheme(
  filled: true,
  fillColor: AppColors.bgLight,
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
    borderSide: BorderSide(color: AppColors.borderColor),
  ),
  hintStyle: TextStyle(color: AppColors.textSecondary),
),
);
}

// Dark theme (optionnel)
static ThemeData darkTheme() {
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    // ...
  );
}
}

```

Changer Palette (Future)

```
// Si tu veux theme sélectionnable par user:
class ThemeProvider extends ChangeNotifier {
  ThemeMode _themeMode = ThemeMode.light;

  void switchPalette(String paletteName) {
    // "green" → AppColors (défaut)
    // "blue" → AppColorsBluePalette
    // "orange" → AppColorsOrangePalette
    notifyListeners();
  }
}
```

```
// Dans main.dart
MaterialApp(
  theme: AppTheme.lightTheme(),
  darkTheme: AppTheme.darkTheme(),
)
```

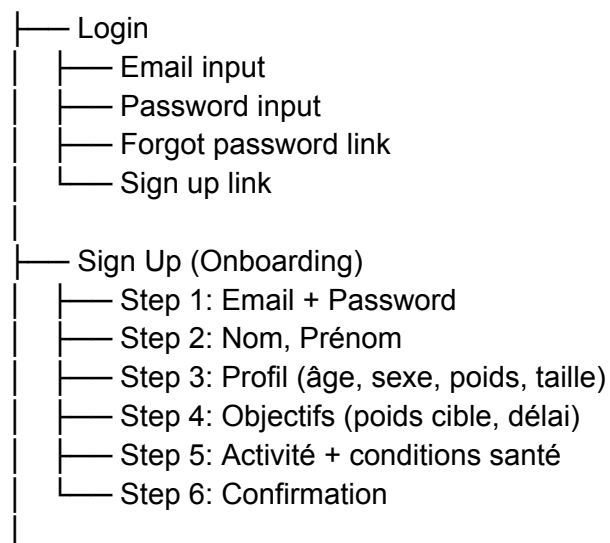
PHASE 1 - DÉTAIL COMPLET

Objectif Phase 1

- ✓ User peut s'inscrire/login
- ✓ Remplit profil + objectifs
- ✓ Ajoute aliments MANUELLEMENT
- ✓ Voit dashboard avec calories/macros
- ✓ Voit tendance sur jour

FEATURES PHASE 1 (Détail)

Écrans:



- └─ Reset Password
 - └─ Email → Code → Nouveau password

Backend:

- Firebase Auth (simple)
- OU Node.js + bcrypt + JWT
- Validation email
- Rate limiting (max 5 tries/5min)

2 PROFIL USER & CALCULS

Data User Stockée:

```
{
  id: "user_123",
  email: "marie@email.com",
  nom: "Marie",
  prenom: "Dupont",
  sexe: "F",
  age: 35,
  poids_kg: 78,
  taille_cm: 165,
  poids_objectif: 68,
  delai_semaines: 12,
  activite: "modere", // sedentaire, leger, modere, actif, tres_actif
  conditions: ["diabete"], // optionnel
  allergies: ["arachide"],
  regime: "omnivore",
  created_at: "2025-01-15"
}
```

Calculs Auto (dans backend):

- BMR (Mifflin-St Jeor) = 1500 kcal
- TDEE (BMR × 1.55 activité) = 2325 kcal
- Besoins caloriques = TDEE ± deficit selon objectif
 - * Si -12kg en 12 semaines = deficit 250 kcal/jour
 - * Donc besoin = 2325 - 250 = 2075 kcal/jour
- Macros recommandées:
 - * Protéines: 2g par kg = 156g/jour
 - * Glucides: 45% calories = 234g/jour
 - * Lipides: 35% calories = 81g/jour

API Return:

```
{
  daily_calories: 2075,
  daily_macros: {
    protein_g: 156,
    carbs_g: 234,
```

```

    fats_g: 81,
    fiber_min: 25
  },
  nutrition_alerts: [{"diabete": limite sucre 50g/jour}]
}

```

3 AJOUTER ALIMENT (Manuel)

Flow:

1. User clique [📄 Texte]
2. Écran "Ajouter aliment"
 - ├─ Recherche aliment (autocomplete)
 - ├─ Requête API OpenFoodFacts:
 - GET /api/v0/products/search?query=poulet
 - Liste "Poulet rôti", "Poulet cru", etc.
 - ├─ Champ quantité (nombre)
 - ├─ 150
 - ├─ Champ unité (dropdown)
 - ├─ gramme / ml / portion / bol / verre / etc
 - ├─ Catégorie repas (dropdown)
 - ├─ Petit-déj / Déj / Goûter / Dîner / Snack
 - └─ Bouton "Ajouter" [vert]
3. Backend valide:
 - Aliment existe dans DB
 - Quantité 1-5000g
 - Unité valide
 - Cherche nutrition dans OpenFoodFacts
4. Retour nutrition (ex):


```

{
  "nom": "Poulet rôti",
  "quantite": 150,
  "unite": "g",
  "calories": 225,
  "protein": 35,
  "carbs": 0,
  "fats": 9,
  "fiber": 0
}

```
5. Confirmation affichée:

"Poulet rôti 150g: 225 kcal, 35g protein"

[Ajouter] [Annuler]

6. Sauvegarde en BD:
meals/{date}/entries

4 DASHBOARD QUOTIDIEN

Layout:

17

Lundi 15 mai

Navigation jour

PERSONNAGE

(Cercle SVG simple)

(% rempli selon calories)

Heureux/Triste selon status

Consommé: 1680 / 2075 kcal

Reste: 395 kcal

81%

Progress bar vert

MACROS:

Protéines: 42g / 156g (27%)

Glucides: 156g / 234g (67%)

Lipides: 56g / 81g (69%)

REPAS DU JOUR:

Petit-Déj

Müesli 50g - 190 kcal

Lait 200ml - 134 kcal

Déjeuner

Poulet 150g - 225 kcal

Riz 150g - 165 kcal

Brocoli 150g - 50 kcal

Snack

Pomme - 95 kcal

Plus (+)

[+ Ajouter] [Plus d'infos]

Boutons

5 ÉDITION ALIMENT

Modal:

×	Éditer Poulet	
Nom: [Poulet rôti]		
Quantité: [150]		
Unité: [gramme ▼]		
Catégorie: [Déjeuner ▼]		
Nutrition (calc auto):		
Calories: 225 kcal		
Protéines: 35g		
Glucides: 0g		
Lipides: 9g		
[Sauvegarder] [Annuler]		

Action:

- Modification = requête PATCH
- Recalcul nutritionnel
- Mise à jour dashboard en temps réel

6 SIMPLE ANALYTICS (Jour)

Swipe up depuis dashboard:

Détails Nutritionnels - Lundi	
Calories: 1680 / 2075	
Protéines: 42g / 156g (✗ bas)	
Glucides: 156g / 234g (ok)	
Lipides: 56g / 81g (ok)	
Fibres: 8g / 25g (✗ bas)	
Sucres: 18g / 50g (ok)	
Sodium: 1200mg / 2300mg(ok)	
SCORE DU JOUR: B-	
✓ Calories bonnes	
⚠ Protéines insuffisantes	
⚠ Fibres trop basses	
✓ Sucres contrôlés	
CONSEIL:	
"Ajoute +30g protéines (œufs, fromage, poisson)"	

[Fermer]

7 BASE DE DONNÉES (Firestore ou PostgreSQL)

Firestore Structure:

```
users/{userId}
├── profile: {email, nom, age, ...}
├── nutrition_targets: {daily_calories, daily_protein, ...}
├── meals/{date}
│   ├── entries/{entryId}
│   │   ├── food_name: "Poulet rôti"
│   │   ├── quantity: 150
│   │   ├── unit: "g"
│   │   ├── calories: 225
│   │   ├── proteins: 35
│   │   ├── carbs: 0
│   │   ├── fats: 9
│   │   ├── meal_type: "lunch"
│   │   ├── source: "openfoods"
│   │   ├── created_at: timestamp
│   │   └── updated_at: timestamp
│   └── daily_summary: {
│       total_calories: 1680,
│       total_protein: 42,
│       total_carbs: 156,
│       total_fats: 56,
│       score: "B-"
│   }
└── }
```

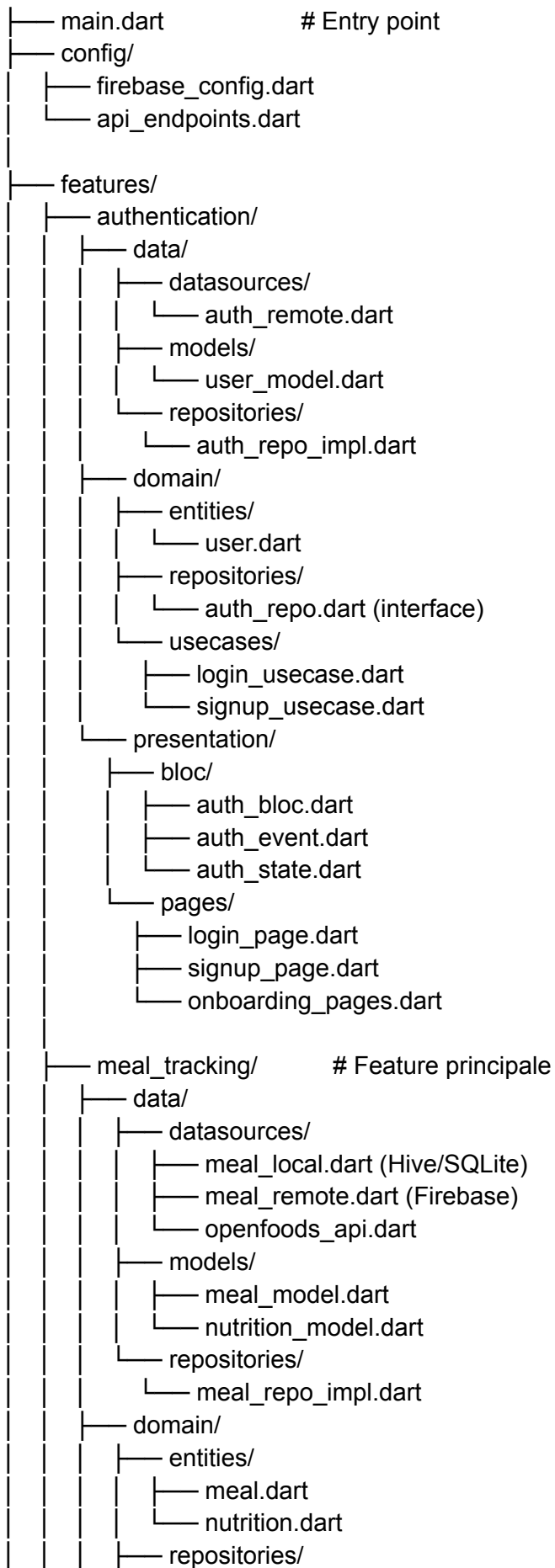
8 SECURITÉ PHASE 1

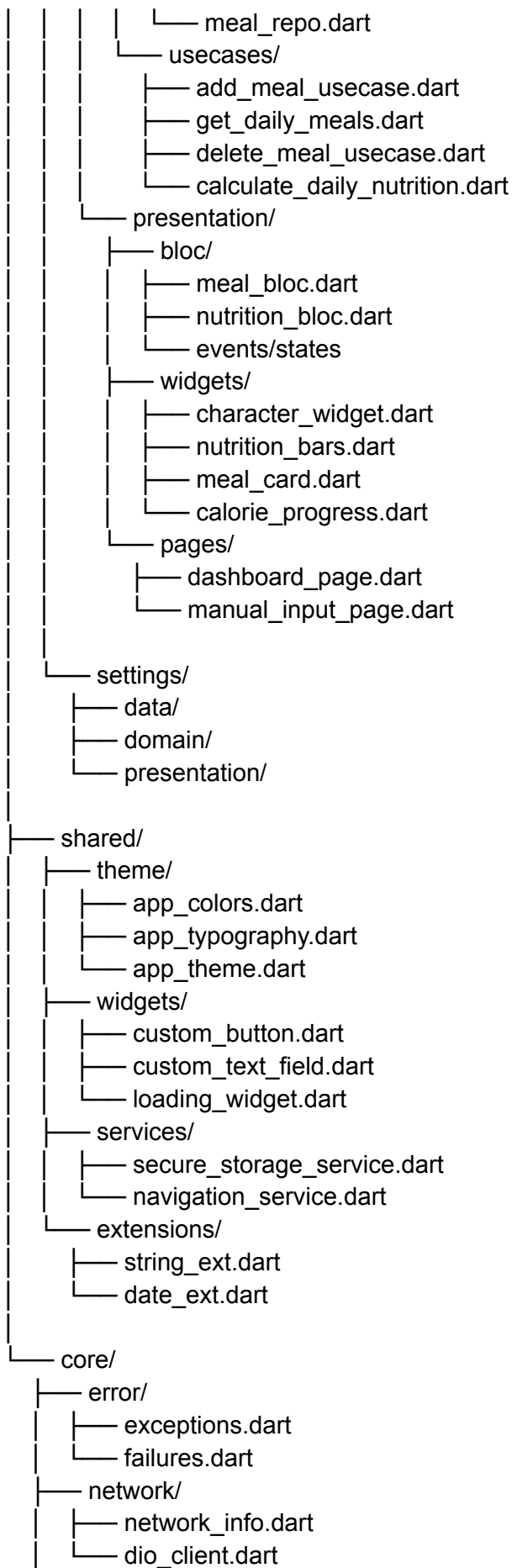
Checklist:

- ✓ HTTPS obligatoire (Firebase auto)
- ✓ Firebase Auth ou JWT + refresh tokens
- ✓ Validation inputs (client + serveur)
- ✓ Rate limiting API (100 req/5min)
- ✓ Pas de données sensibles en logs
- ✓ Firestore Rules (user voit que ses data)
- ✓ Mot de passe min 8 chars, maj+min+num
- ✓ Tokens expiration (15min access, 7j refresh)

ARCHITECTURE CODE PHASE 1

lib/





```
└─ constants/  
  └─ app_constants.dart
```

DELIVERABLES PHASE 1

Git Repo Structure:

```
nutritrack/  
├─ 📱 lib/ (code Flutter)  
├─ 🔒 backend/ (Node.js OU Firebase)  
├─ 📄 pubspec.yaml (dépendances)  
├─ 📖 README.md (setup instructions)  
├─ 🔧 .env.example (variables env)  
├─ 📐 ARCHITECTURE.md (doc structure)  
├─ 🧪 test/ (unit tests essentiels)  
└─ 🎨 design_system/ (couleurs, typo constantes)
```

Livrables App:

- ✓ App installable (.apk Android, .ipa iOS)
- ✓ Backend déployé (Firebase ou Node.js)
- ✓ Database configurée (Firestore ou PostgreSQL)
- ✓ Documentation setup
- ✓ Code review friendly (clean, commenté)

SUCCESS CRITERIA PHASE 1

- ✓ User crée compte + profil
- ✓ User ajoute 5+ aliments
- ✓ Dashboard affiche calories/macros corrects
- ✓ Édition/suppression fonctionne
- ✓ Data synchro cloud
- ✓ App stable 0 crashes
- ✓ Temps load < 2s
- ✓ Sécurité validée (tokens, HTTPS, etc.)
- ✓ Code review clean (architecture respectée)

=== VITAE - APPLICATION SUIVI NUTRITIONNEL INTELLIGENT ===
=== PHASE 1: MVP COMPLET ===

CONTEXTE GLOBAL:

Développer une application mobile Flutter complète (iOS + Android) de suivi alimentaire et nutritionnel avec tableau de bord intelligent et personnage animé comme feedback visuel. L'app aide utilisateurs à prendre conscience de leur alimentation et atteindre objectifs santé.

SECTION 1: STACK TECHNOLOGIQUE

Frontend:

- ✓ Framework: Flutter (Dart)
- ✓ Architecture: Clean Architecture (Data / Domain / Presentation)
- ✓ State Management: Riverpod (injection + caching)
- ✓ Injection Dépendances: GetIt
- ✓ Patterns: BLoC pattern (Cubit recommandé)

Backend:

- ✓ Phase 1: Firebase (Firestore + Auth + Storage)
(Possibilité migration Node.js Phase 3+ si nécessaire)
- ✓ Base données: Firestore (NoSQL)
- ✓ Authentification: Firebase Auth
- ✓ Variables env: flutter_dotenv

APIs Externes:

- ✓ OpenFoodFacts API (base aliments nutrition)
- ✓ Google ML Kit (optionnel détection objets Phase 2)

Stockage Local:

- ✓ Secure Storage: flutter_secure_storage (tokens)
- ✓ Cache: Hive (aliments favoris, derniers)
- ✓ Préférences: shared_preferences (settings légers)

UI/Animations:

- ✓ Design System: Material Design 3
- ✓ Icons: Material Design Icons
- ✓ Animations: Lottie (personnage SVG)
- ✓ Graphiques: FL Chart (macros radar, calories trend)

Qualité Code:

- ✓ Analyse: dart analyze
- ✓ Format: dart format
- ✓ Tests: integration tests essentiels
- ✓ Linting: custom_lint rules

SECTION 2: DONNÉES UTILISATEUR & CALCULS

PROFIL USER (Collecté Onboarding):

```
{  
  uid: string (Firebase ID),  
  email: string,  
  nom: string,  
  prenom: string,  
  sexe: "M" | "F" | "Autre",  
  age: int,
```

```

poids_kg: double,
taille_cm: int,
poids_objectif_kg: double,
delai_semaines: int,
niveau_activite: "sedentaire" | "leger" | "modere" | "actif" | "tres_actif",
conditions_sante: List<string>, // ["diabete", "hypertension"]
allergies: List<string>,
regime: string, // omnivore, vegetarien, vegan
cuisines_preferees: List<string>,
created_at: timestamp
}

```

CALCULS AUTOMATIQUES (Backend):

Formule BMR (Mifflin-St Jeor):

- Homme: $(10 \times \text{poids}) + (6.25 \times \text{taille}) - (5 \times \text{age}) + 5$
- Femme: $(10 \times \text{poids}) + (6.25 \times \text{taille}) - (5 \times \text{age}) - 161$

Facteurs Activité:

- Sédentaire: 1.2
- Léger: 1.375
- Modéré: 1.55
- Actif: 1.725
- Très actif: 1.9

$\text{TDEE} = \text{BMR} \times \text{facteur activit }$

Besoin Calorique Ajust :

- Perte poids: $\text{TDEE} - 250 \text{ kcal}$ (pour -250g/semaine)
- Maintien: TDEE
- Prise: $\text{TDEE} + 250 \text{ kcal}$

Seuils Nutritionnels (personnalis s):

- Prot ines: 1.6-2.2g par kg poids
- Glucides: 45-65% des calories
- Lipides: 20-35% des calories
- Fibres: 25-30g minimum
- Sucres: 50g maximum recommand 
- Sodium: 2300mg maximum
- (Ajust s si conditions sant : diab te → sucre baiss )

API Retour:

```

{
  daily_calories: 2075,
  daily_macros: {
    protein_g: 156,
    carbs_g: 234,
    fats_g: 81,
    fiber_min: 25
  }
}

```

```
},  
nutrition_alerts: {  
  "diabete": "Limiter sucres à 50g/jour",  
  "hypertension": "Sodium max 1500mg/jour"  
}  
}
```

SECTION 3: FEATURES PHASE 1 (DÉTAIL)

FEATURE 1: AUTHENTICATION & ONBOARDING

Écrans:

1. Login

- Email input (validation format)
- Password input (masquage)
- Forgot Password link
- Sign Up button
- Validation: Email exist + password correct
- Error handling: afficher erreurs user-friendly

2. Sign Up (6 étapes - Wizard)

Step 1: Données de base

- Prénom (text input)
- Nom (text input)
- Email (text input)
- Validation: email unique

Step 2: Sécurité

- Password (text input, min 8 chars)
- Confirm password
- Password strength indicator (rouge → vert)
- Requirements visibles: maj, min, chiffre

Step 3: Profil Physique

- Sexe (radio buttons: M/F/Autre)
- Age (number input, 13-120)
- Poids (double input, 30-300kg)
- Taille (number input, 100-250cm)
- Affichage BMI en temps réel

Step 4: Objectifs

- Poids objectif (input, validation > poids actuel ou <)
- Délai en semaines (input, 4-52)
- Calcul déficit/surplus affiché
- Réalisme check: "Objectif ambitieux mais réalisable"

Step 5: Activité & Santé

- Niveau activité (dropdown/radio, 5 options)
- Liste checkboxes conditions santé
- Liste inputs allergies (add/remove)
- Dropdown régime alimentaire

Step 6: Confirmation

- Résumé toutes infos
- Bouton "Éditer" par section
- Bouton "Confirmer création"

3. Reset Password

- Email input
- Validation firebase
- Message "Email envoyé"
- Lien dans email avec token

Sécurité:

- ✓ Validation client (format email, password strength)
- ✓ Firebase Auth gère hachage + stockage sécurisé
- ✓ Tokens localStorage sécurisé (flutter_secure_storage)
- ✓ Session 30 jours automatic logout
- ✓ Refresh token automatique

FEATURE 2: DASHBOARD QUOTIDIEN (Écran Principal)

Layout Principal:

 Lundi 15 mai		[◀] [▶]	← Navigation jour
 PERSONNAGE			
(Cercle SVG animé)			
(% remplissage calories)			
Animations: heureux/triste/malade			
Consommé: 1680 kcal			
Objectif: 2075 kcal			
Reste: 395 kcal			
 81%			
MACRONUTRIMENTS:			
	Protéines: 42g / 156g (27%)		
	Glucides: 156g / 234g (67%)		
	Lipides: 56g / 81g (69%)		

REPAS AJOUTÉS:		
 Petit-Déjeuner		
└─ Müesli 50g	190 kcal	
└─ Lait 200ml	134 kcal	
 Déjeuner		
└─ Poulet rôti 150g	225 kcal	
└─ Riz blanc 150g	165 kcal	
└─ Brocoli 150g	50 kcal	
Plus (+)		
[+ Ajouter aliment] [Plus d'infos]		

Interactions Dashboard:

- Tap date prev/next: change jour (accès historique)
- Tap personnage: affiche conseil motivant
- Swipe up: affiche analytics complètes jour
- Long press aliment: options édition/suppression
- Swipe left aliment: delete with confirmation

Détails Personnage:

- SVG Lottie simple (style cartoon, stylisé)
- Animations:
 - * 0-50% calories: blanc/gris neutre
 - * 50-85% calories: vert heureux
 - * 85-100% calories: orange neutre
 - * >100% calories: rouge inquiet
- Messages contextuels:
 - * "Bon équilibre!" (macros OK)
 - * "Attention sucres!" (si trop sucre)
 - * "Protéines insuffisantes" (si <80%)

Détails Barres Macros:

- 3 barres horizontales (protéines, glucides, lipides)
- Code couleur:
 - * Vert: dans zone cible
 - * Orange: 10-20% hors zone
 - * Rouge: >20% hors zone
- Animation remplissage smooth (animationDuration: 500ms)
- Tap barre: affiche détail + suggestions

FEATURE 3: AJOUTER ALIMENT (MANUEL - PHASE 1)

Écran Manuel Input:

+ Ajouter un aliment	
Rechercher aliment	
	
Suggestions: Poulet Riz ...	
Quantité	
	150
Unité	
[Gramme ▼]	
Options: g, ml, portion, bol, etc	
Repas	
[Déjeuner ▼]	
Nutrition trouvée (auto-calc):	
Calories: 225 kcal	
Protéines: 35g Glucides: 0g	
Lipides: 9g Fibres: 0g	
[+ Ajouter] [Annuler]	

Flow Détaillé:

1. User tape "pou" dans recherche
→ Autocomplete appelle OpenFoodFacts API:
GET <https://world.openfoodfacts.org/api/v0/products/search?query=pou>
→ Retour:

```
[  
  {name: "Poulet rôti", code: "123"},  
  {name: "Poulet cru", code: "456"}  
]
```


→ Affiche liste dropdown
2. User sélectionne "Poulet rôti"
→ Récupère id produit OpenFoodFacts
3. User rentre quantité "150" et unité "gramme"
→ Validation: $1 \leq qty \leq 5000$
4. Backend requête nutrition:
GET [/api/nutrition/{product_id}?quantity=150&unit=g](#)

→ OpenFoodFacts API retourne:

```
{
  "name": "Poulet rôti",
  "calories": 225,
  "protein": 35,
  "carbs": 0,
  "fats": 9,
  "fiber": 0,
  "sodium": 50
}
```

5. Frontend affiche nutrition

6. User clique [+ Ajouter]

- Validation serveur
- Sauvegarde Firestore: meals/{date}/entries/{id}
- Recalcul daily_summary
- Dashboard refresh automatique (real-time listener)

Error Handling:

- Aliment non trouvé: "Aliment non trouvé. Essaie autres mots clés."
- Quantité invalide: "Quantité entre 1 et 5000"
- Connexion offline: "Cache local activé. Sync dès que possible."

Favoris Rapides:

- Les 10 derniers aliments ajoutés = boutons rapides
- Click 1x = ajoute même quantité dernière utilisation
- Swipe favoris = sélection/suppression

FEATURE 4: ÉDITER / SUPPRIMER ALIMENTS

Édition (Modal):

× Éditer Poulet rôti	
Quantité:	
[150] gramme	
Repas:	
[Déjeuner ▼]	
Nutrition (recalculée):	
Calories: 225 kcal	
Protéines: 35g	
Glucides: 0g	

Lipides: 9g	
[Sauvegarder] [Annuler]	

Actions:

- Change quantité → nutrition recalculée temps réel
- Clique [Sauvegarder] → PATCH /meals/{date}/{entryId}
- Dashboard update immédiatement (Firestore listener)
- Undo option (toast "Modifié" avec bouton undo 3s)

Suppression:

- Swipe left aliment → affiche [Supprimer]
- Tap [Supprimer] → confirmation "Supprimer cet aliment?"
- Click confirm → DELETE /meals/{date}/{entryId}
- Toast "Aliment supprimé" + undo 3s

FEATURE 5: ANALYTICS SIMPLE (Jour)

Swipe up depuis dashboard:

← Détails Lundi 15 mai	
CALORIC INTAKE:	
1680 / 2075 kcal (81%)	
Reste: 395 kcal	
MACRONUTRIMENTS DÉTAILS:	
Protéines: 42g / 156g (✗ -75%)	
Glucides: 156g / 234g (✓ ok)	
Lipides: 56g / 81g (✓ ok)	
Fibres: 8g / 25g (✗ -68%)	
INDICATEURS SANTÉ:	
Sucres: 18g / 50g (✓)	
Sodium: 1200mg / 2300mg (✓)	
Cholestérol: 45mg (ok)	
SCORE DU JOUR: B-	
(Bon équilibre général)	
CONSEILS VITAE:	
"Ajoute +80g protéines aujourd'hui	
pour atteindre ton objectif.	
Suggestions: œufs, fromage, poisson"	

[Fermer]

Logique Scoring:

- Calories OK (80-120% objectif): +40 pts
- Macros OK (protéines 80-120%): +30 pts
- Santé OK (sucres, sodium): +30 pts
- Score total 90-100: A
- Score total 80-89: B
- Score total 70-79: C
- Score total <70: D/F

Couleurs Score:

- A (90+): Vert
- B (80-89): Orange
- C-D (<79): Rouge

FEATURE 6: SYSTÈME DE SÉCURITÉ PHASE 1

Authentication:

- ✓ Firebase Auth gère hashage + sessions
- ✓ Tokens (access + refresh):
 - Access: 15 minutes
 - Refresh: 7 jours
- ✓ Token stocké: flutter_secure_storage (chiffré OS)
- ✓ Logout automatique après inactivité 30 jours

Validation:

- ✓ Client-side:
 - Email format validation
 - Password 8+ chars, maj+min+chiffre
 - Quantités 1-5000
- ✓ Server-side:
 - Validation stricte tous inputs
 - SQL injection protection (Firestore/ORM)
 - Rate limiting: 100 requêtes/5 minutes

Data Privacy:

- ✓ Données sensibles (poids, conditions):
 - Chiffrement transit (HTTPS)
 - Accès restreint (user voit que SES data)
- ✓ Photos repas: Pas stockées Phase 1
- ✓ Logs serveur: Pas de données sensibles
- ✓ RGPD: Export/Delete data possible (future feature)

Firestore Security Rules:

```
rules_version = '2'; service cloud.firestore { match /databases/{database}/documents { //
Only user can read/write their own data match /users/{userId} { allow read, write: if
request.auth.uid == userId; } match /users/{userId}/meals/{document=} { allow read, write:
if request.auth.uid == userId; } // Public: food nutrition data (read-only) match /foods/{foodId}
{ allow read: if true; allow write: if false; } }
```

SECTION 4: DESIGN SYSTEM & INTERFACES

PALETTE COULEURS (Dynamique via Constantes):

```
app_colors.dart:
```dart
class AppColors {
 // Primaires
 static const primaryColor = Color(0xFF27AE60); // Vert
 static const primaryDark = Color(0xFF1E8449);
 static const primaryLight = Color(0xFFA9DFBF);
 static const primaryPale = Color(0xFFD5F4E6);

 // Accent
 static const accentColor = Color(0xFFFF39C1); // Orange
 static const accentDark = Color(0xFFC17C1B);
 static const accentLight = Color(0xFFFFCE5CD);

 // Alerts
 static const errorColor = Color(0xFFE74C3C); // Rouge
 static const errorDark = Color(0xFFA93226);
 static const errorLight = Color(0xFFFFADB8);

 // Neutrals
 static const textPrimary = Color(0xFF2C3E50);
 static const textSecondary = Color(0xFF95A5A6);
 static const bgLight = Color(0xFFFFF8F9FA);
 static const bgDark = Color(0xFFFFFFFF);

 // Charts
 static const chartProtein = Color(0xFF3498DB);
 static const chartCarbs = Color(0xFFFF39C1);
 static const chartFats = Color(0xFFE74C3C);
 static const chartFiber = Color(0xFF27AE60);
}
```
```

TYPOGRAPHIE:

Fonts:

- Inter (headings, buttons): weights 600, 700
- Poppins (body): weights 400, 500
- Source: google_fonts package

Échelle:

- H1 (Display): 30px / 38px line-height
- H2 (Heading): 24px / 32px
- H3 (Subheading): 18px / 26px
- Body: 16px / 24px
- Label: 14px / 20px
- Caption: 12px / 16px

Principes:

- Espaces blancs généreux (30% écran vide)
- Max 3-4 éléments principaux par écran
- Une action principale (CTA) par page
- Padding écran: 16px côtés, 24px haut/bas
- Border radius: 12px (standard), 8px (petits)
- Shadows: elevation 2-4 (subtle)

ICONES:

- Material Design Icons (package flutter)
- Fallback: Feather Icons

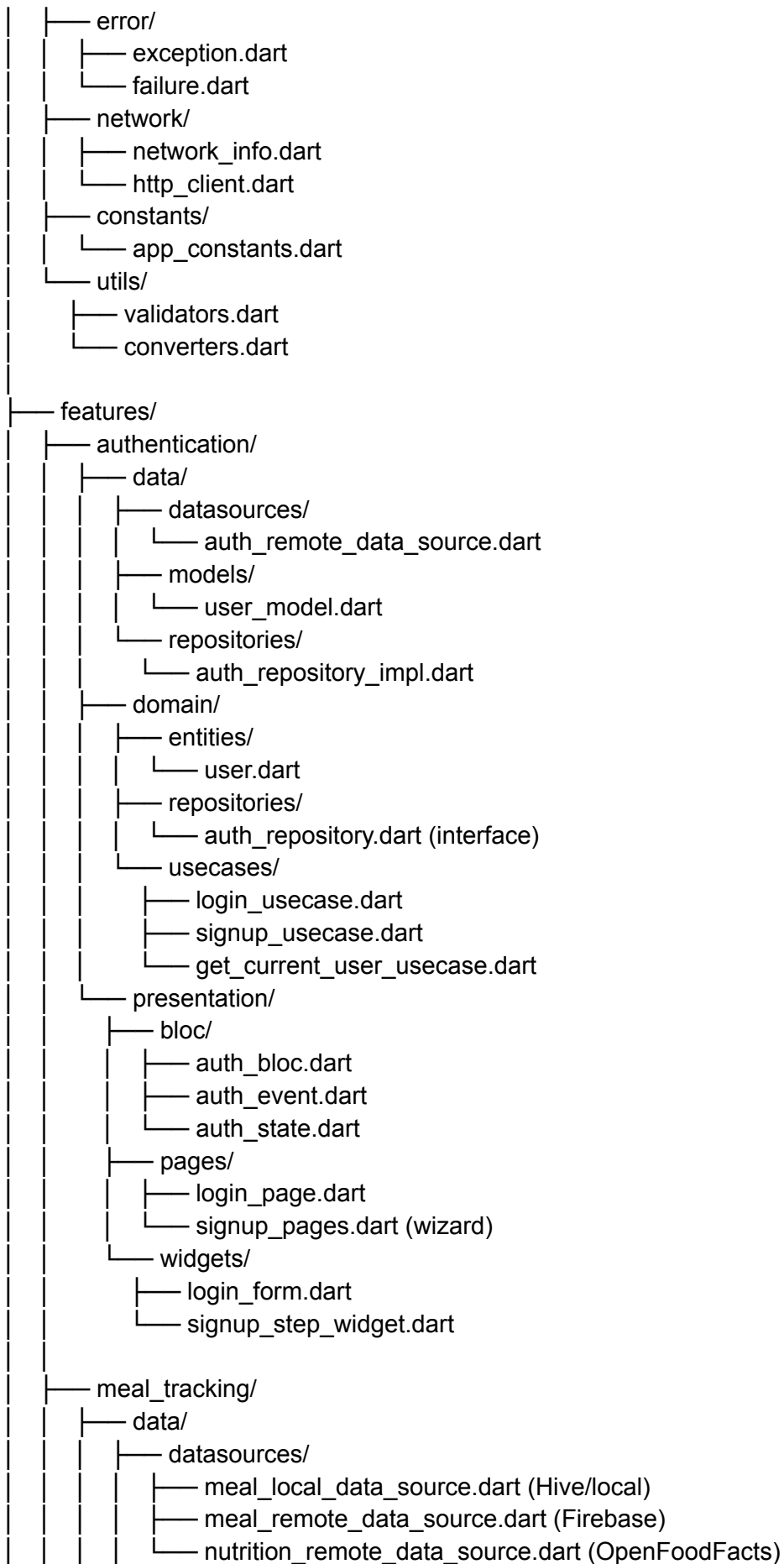
ANIMATIONS:

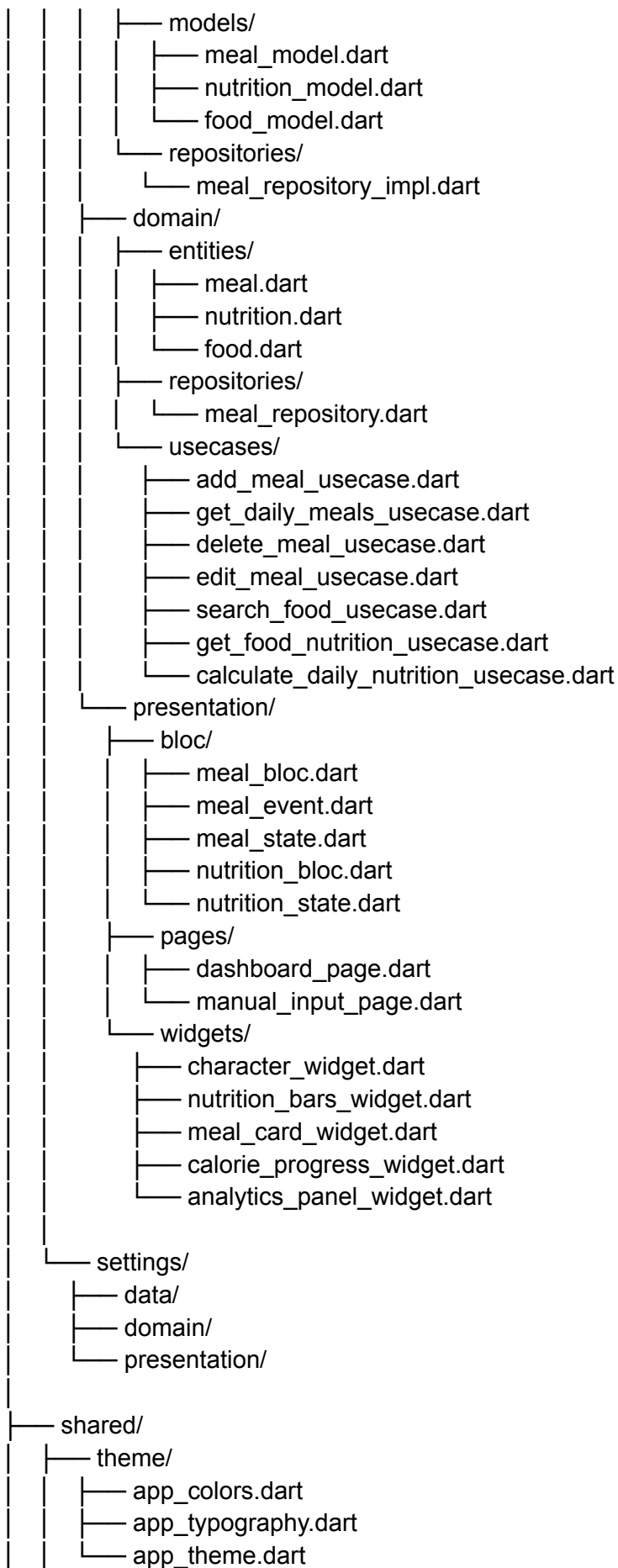
- Personnage: Lottie SVG (simple, fluide, 60fps)
- Transitions: fade 200ms, slide 300ms
- Progress bars: smooth fill (500ms)
- Boutons: scale 100ms on press

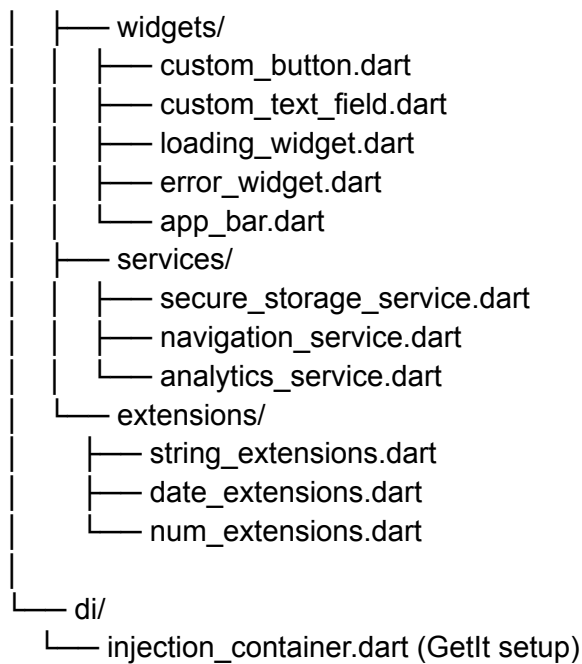
SECTION 5: ARCHITECTURE CODE CLEAN

Structure Package (Clean Architecture):

```
lib/  
├── main.dart  
├── config/  
│   ├── firebase_config.dart  
│   └── app_config.dart  
└── core/
```







Key Principles:

- ✓ Séparation stricte Data/Domain/Presentation
- ✓ Repositories pattern (interfaces domain, impl data)
- ✓ Use Cases = business logic
- ✓ BLoC/Cubit = state management
- ✓ No imports from Presentation to Data (dependency rule)
- ✓ Constants dynamiques (theme, API endpoints)
- ✓ Dependency injection (GetIt)
- ✓ Error handling unifié (Failures, Exceptions)
- ✓ Comment les entités/modèles
- ✓ Tests unitaires pour logique critique

SECTION 6: QUALITÉ & SÉCURITÉ

Code Review Checklist:

- ✓ Architecture Clean respectée
- ✓ Pas de logique business en UI
- ✓ Services injectés (pas new direct)
- ✓ Error handling complet (try/catch, bloc states)
- ✓ Validation inputs (client + server)
- ✓ Pas de données sensibles en logs
- ✓ Tests unitaires couvrent 70%+ logique métier
- ✓ Constantes pour tous les values (colors, strings)
- ✓ Commentaires JSDoc sur méthodes publiques
- ✓ Analyse statique passante (dart analyze)

Sécurité:

- ✓ Tokens stockés secure_storage (chiffré)
- ✓ HTTPS obligatoire (Firebase auto)
- ✓ Firestore Rules restrictives (user voit data)
- ✓ Validation server-side stricte
- ✓ Rate limiting API
- ✓ Pas de credentials en code
- ✓ Variables env (.env.example fournir)
- ✓ Logout efface tokens locaux

Testing Phase 1:

- ✓ Unit tests usecases (logique nutrition)
- ✓ Unit tests repositories (mocks)
- ✓ Integration tests dashboard (Firebase emulator)
- ✓ Widget tests composants critiques
- ✓ Minimum coverage 70% logique business

SECTION 7: DESIGNS VISUELS

Inspiration Visuelle:

- Base structure: LifeFit app UI Kit (50 screens reference)
- Style épuré: Coffee Minimalist app UI
- Cohérence: Kupa food app (navigation, interactions)
- Palette: Vert naturel #27AE60 + Orange #F39C12
- Espaces blancs: 30% écran vide minimum

Écrans Principaux à Développer:

1. Login screen (simple, épuré)
2. Sign Up wizard (6 étapes, progressbar)
3. Dashboard quotidien (personnage animé, macros, liste repas)
4. Manual input screen (recherche aliment, quantité)
5. Edit meal screen (modal, nutrition temps réel)
6. Analytics slide-up panel (détails jour)
7. Settings screen (profil, édition)

Animations:

- Personnage Lottie: simple style cartoon
- Dashboard transitions: fade 200ms
- Meal cards: slide in 300ms
- Progress bars: smooth fill 500ms
- Loading: spinner simple Material

SECTION 8: DÉPLOIEMENT & LIVRAISON

Build & Distribution:

Android:

- ✓ Build release APK: flutter build apk --release
- ✓ Optimisations: R8 enabled, split-apks
- ✓ Sizing: <50MB target

iOS:

- ✓ Build release IPA: flutter build ios --release
- ✓ Provisioning: configure certificates
- ✓ Sizing: <100MB target

Testflight/Beta:

- ✓ Firebase App Distribution (easy)
- ✓ Ou TestFlight iOS, Google Play Internal Testing

Production:

- ✓ Google Play Store submission
- ✓ Apple App Store submission
- ✓ Documentation release notes

Backend Deployment:

- ✓ Firebase (Firestore, Auth, Hosting): auto
- ✓ Ou Heroku/Railway (Node.js future)

SECTION 9: LIVRABLE FINAL

Git Repository Structure:

```
nutritrack/
├── lib/           # Code Flutter
├── test/         # Tests
├── android/      # Config Android
├── ios/          # Config iOS
├── pubspec.yaml  # Dépendances
├── pubspec.lock  # Lock file
├── .env.example  # Variables env template
├── README.md     # Setup instructions
├── ARCHITECTURE.md # Doc architecture
├── CODE_STYLE.md # Style guide
├── SECURITY.md    # Security guidelines
├── PHASE2_ROADMAP.md # Next steps
└── design_system/ # Images designs
```

Documentations Fournir:

- ✓ README: instructions setup (Flutter, Firebase)
- ✓ ARCHITECTURE.md: clean architecture explanation
- ✓ CODE_STYLE.md: conventions (naming, formatting)
- ✓ SECURITY.md: sécurité best practices
- ✓ API.md: endpoints Firebase utilisés
- ✓ DATABASE.md: Firestore structure schema

Code Quality:

- ✓ Zéro erreurs dartanalyzer
- ✓ Format: dart format applied
- ✓ Comments: JSDoc sur méthodes publiques
- ✓ Constants: Tous les values en consts
- ✓ Tests: 70%+ coverage logique business

Success Criteria Phase 1:

- ✓ User peut créer compte complet
- ✓ Profil calculé correctement (BMR/TDEE)
- ✓ Ajouter aliments manuellement fonctionne
- ✓ Dashboard affiche calories/macros corrects
- ✓ Édition/suppression sans bug
- ✓ Data synchro Firebase seamless
- ✓ App stable (0 crashes 1000 interactions)
- ✓ Load time <2 secondes
- ✓ Code review clean
- ✓ Tests passent 100%

SECTION 10: QUESTIONS CLARIFICATION

1. Langue: App en français? (UI + strings)
2. Platform: iOS + Android simultanément
3. Dark mode: Implémenter Phase 1
4. Offline-first: Priorité (app marche offline)? Non
5. Multi-language: Future
6. Backend: Firebase Phase 1 confirmé
7. Notifications: Push Phase 1 ou Phase 2
